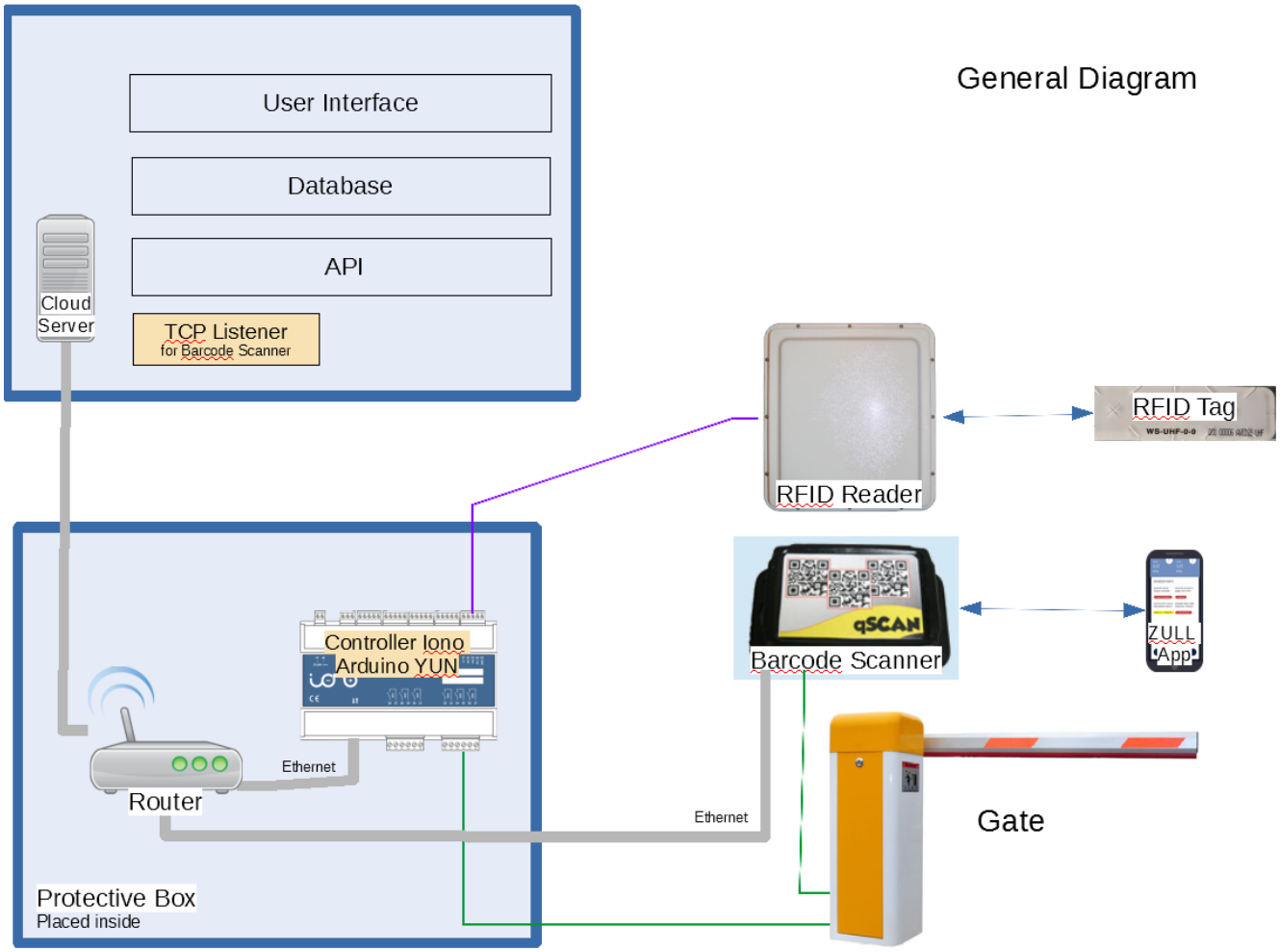
Zuul: Setup the Gate Control System

The main components of Zuul gate control system



Protective box

Near the gate, a protective box should be installed.
It contains the *Router*, the *Controller* and all power supplies.

Router

Use any *Ethernet Router* connected to the internet.

Gate Mechanism

Any gate or barrier that can be triggered with a relay.
It should have a built-in IR sensor to prevent closing the gate while a car is in transit.

Barcode Scanner: qScan TCP

It should be mounted within reach of any driver stopped in front of the gate.
It is connected using an ethernet cable to the *Router*, and it's relay is connected to the *Gate Mechanism*.
It's power supply is placed inside the *Protective Box.*
It reads bar-codes generated by the *Zuul* APP.
It connects over the *Internet* to the *TCP Listener* running on the *Cloud Server* to verify codes and it triggers the *Gate Mechanism* when good codes are detected.
It will not work if offline, without internet connection.

RFID Reader: Awid LR-2000 or Maxtek Q-1000

It should be mounted in front of the gate at an angle that limits it's range to only one car.
It is connected to the *Controller* and it's power supply is placed inside the *Protective Box.*
It detects *RFID Tags* placed inside incoming cars and sends the *RFID Tag* code to the *Controller.*

Controller: Iono Arduino YUN

It is mounted inside the box.
It is connected using an ethernet cable to the *Router*, and it's relay is connected to the *Gate Mechanism*, in parallel with the *Barcode Scanner* relay.
It can share the power supply with the *RFID Reader*.
It receives data from the *RFID Reader* and it triggers the *Gate Mechanism* when good codes are detected.
It stores a list of allowed *RFID Tag* codes and keeps an access log.
It connects over the *Internet* to the *Cloud Server* to save the access log and to download the latest allowed codes.
It continues to work even offline.

TCP Listener

It is a software running on the *Cloud Server* that accepts socket connections from all Zull *Barcode Scanners.*
It receives bar-codes and forwards them to the *Zuul API* for validation, then sends the appropriate command back to the scanner.
It is written in Bash and uses the Linux TCP utility Socat.

Controller Firmware and Software

The Iono Arduino YUN controller is a micro-controller board with OpenWrt Linux.
The *Controller* is programmed with a compiled *Firmware*, a few Bash *Scripts* and by changing a few OpenWRT settings.
The *Firmware* is written in C++ using Arduino IDE

1. Initial Setup

Configuring and testing the devices

Before on-site install, the *Barcode Scanner*, the *RFID Reader* and the *Controller* need to be setup, configured and tested. On your workbench, wire them as described in the “Simple Wiring Diagram”

Initial Barcode Scanner setup

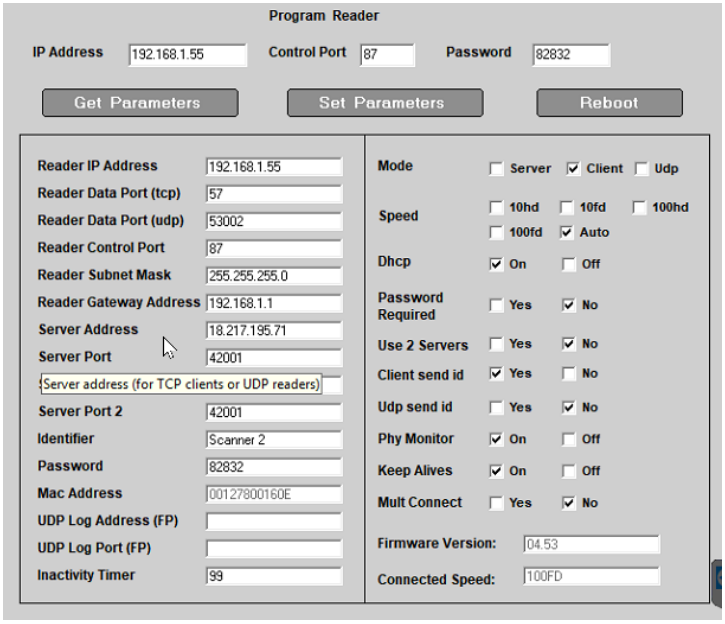
Each scanner should be assigned a UNIQUE identification number N.
As a convention, N equals the auto-generated ID from the Zuul back-end/database. So, first, add a Scanner in the back-end and take note of the new "ID" = N
Write that number with a marker on the back of the scanner.
The Server Port Number is calculated using the formula {42000+ceil(N/5)} (=42001 for the first 5 devices)
The scanner Identifier should be "Scanner N"
The numeric Password will be calculated using the formula ((41414+N)*N) - only the first 5 digits.

Configure the Scanner:

- connect the scanner to the same router as the laptop
- use Advanced IP Scanner to find the scanner IP address
- install IBC TCP/IP Utility Software "Tcp Utility V4.4"
- run TCP/IP Utility and go to "Program Reader" page
- Write the scanner IP address and Click [Get Parameters]
- Set parameters:

```
Server 1&2 Address                      18.217.195.71
Server 1&2 Port Number                  42001
Identifier                              Scanner N
Password                {first 5 digits from ((41414+N)*N)}
Inactivity Timer                        99
MODE                                    TCP Client
DHCP                                    on
Password Required                       off
Clinet Send ID                  on
Phy monitor                             on
Keep Alives                             on
```

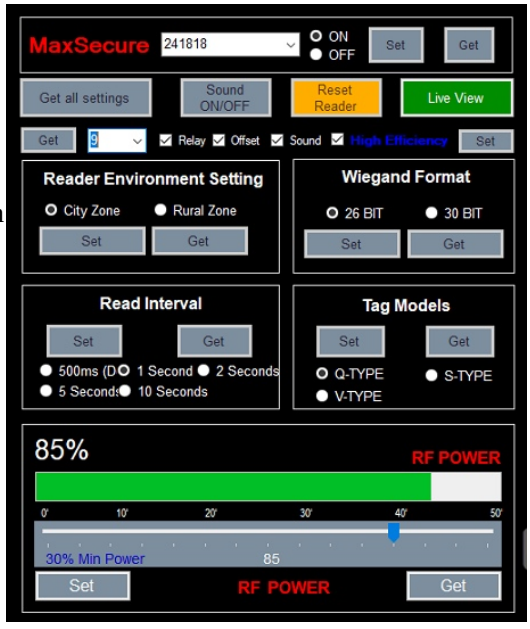
- Click [Set Parameters] then [Reboot]



Initial RFID Reader Setup

- AWID readers require no initial setup, but their range and power can be adjusted by connecting them directly to a laptop using a serial to USB adapter and running AWID Setup software
- Maxtek readers require some configuration changes by connecting them directly to a laptop using a USB Mini-A cable and running the Maxtek Q series programmer software:

```
MaxSecure               241818 ON
Get                     9
Wiegand Format          26bit
Read Interval           1 second
Tag Model               Q-TYPE
```

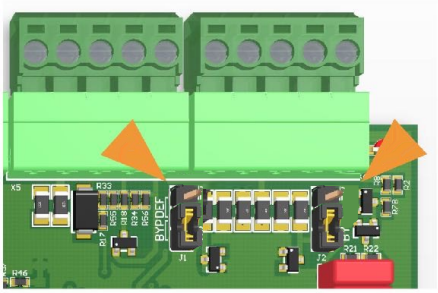


Initial Controller Setup - Iono Arduino YUN

Add the controller to the Zuul Database:

- in the Zuul admin interface add a new controller ; uniqueId is the serial number "F12345678"
- copy or make a note of the new ***secretKey*** hat was generated
- depending on the reader that will be connected, write with a marker on the label: **A** for Awid and **M** for Maxtek

Move the jumpers



- Unplug the Controller and open it:
- Use a flat screwdriver, a bit bigger, insert it's tip just a bit above the green connectors, put your thumb on the blue label, and gently pull up the plastic cover
- Move jumpers from the top right corner J1 and J2 to the BYP position

Change Settings

- connect the laptop and the controller to the same router using Ethernet cables
- use Advanced IP Scanner to find the controller IP address [newIp]
- Browse http://[newIp] - pass: arduino
- set as name the serial number: Yún name = "F12345678"
- set root password = [*secretKey*YUN]
- disable WiFi permanently
- reboot

Upload the Bash scripts

- from firmware.zip copy the files inside /root to the controller
- connect to scp://root:[*secretKey*YUN]@[newIp]
- copy firmware.zip/root/control.sh to /root
- test the script:

```
> chmod 0750 /root/*.sh
> /root/control.sh allcodes
> reset-mcu && rm /tmp/access.log && rm /tmp/*.buf
```

Create Cron jobs

- browse to http://[newIp]/luci/admin/system/crontab
- add in System > Scheduled Tasks

```
30 4 * * * /root/control.sh daily &> /tmp/control.daily.out
20 */1 * * * /root/control.sh hourly &> /tmp/control.hourly.out
@reboot /root/control.sh reboot &> /tmp/control.reboot.out
```

- click [Submit] ; Manually restart the cron service if the crontab file was empty before editing :
- go to System > Startup – *CRON* and click [Restart]

Compile the firmware

- extract the firmware source code from firmware.zip/src/control
- edit Control.ino with Arduino IDE and set

```
#define UniqueID "F12345678"         // The controller Serial Number, same as hostname
#define SecretKey "*secretKey*"      // The controller SecretKey
#define READER_TYPE 3                // 1=default 2=AWID 3=Maxtek
```

- Click Sketch > Export compiled binarry
- Rename *Control.ino.with_bootloader.hex* to *control_F12345678_MAXTEK.hex*
- Upload the firmware to the controller by browsing to http://[newIp]
- After the reboot you should hear the relays clicking and in /tmp there will be cache and log files

2. On-Site install

Router Setup

The router used in the final on-site setup should:

- have DHCP Enabled
- be Connected Continuously to the internet
- reconnect automatically
- reboot daily

3. Cloud Server Setup

Install the TCP Listener:

- Open firewall ports 42001 -> 42005
- Unpack archive `listener.zip` in a temporary folder /tmp

```
> chmod 0700 /tmp/listener/install/install
> sudo /tmp/listener/install/install
```

- Edit the listener config file `/opt/listener/conf/listen.conf`

This will install the listener in `/opt/listener` as user `listener`, and set file permissions.

The scanners are using server IPv4 address `18.217.195.71`. If this address will ever change, ALL scanners will need to be reconfigured using IBC TCP/IP Utility Software

About The TCP Listener

It allows Barcode Scanners that are configured as TCP Clients to connect and maintain a persistent connection.
It is installed in `/opt/listener` and runs in background. It is started by a cron job as user `listener`. If it crashes, it restarts automatically.
It listens on port 42001 and 42002 and accepts multiple clients. Use max 5 clients (devices) /port to be safe.
The listener will only accept connections from scanners with a valid Identifier "Scanner N" and only one connection per identifier.
The connections are monitored every 2 minutes, so in case of a network error, scanners will be forced to reconnect.

TCP Listener Administration

See a list of connected scanners:

```
> sudo find '/opt/listener/run/' -type f -printf "%f\n" -nowarn | grep .pid
```

Disconnect all scanners (they will reconnect in max 3 minutes)

```
> sudo killall socat
```

See the last data sent by scanner 1

```
> sudo tail '/opt/listener/log/Scanner1.log'
```

Expanding - adding more scanners:

```
- first 5 scanners Identifier should be "Scanner 1" "Scanner 2" "Scanner 3" "Scanner 4" "Scanner 5"
- first 5 scanners should use port 42001 ;
- the next 5, port 42002
- the next 5, port 42003 ( + the listener needs to be configured to listen on  42003 )
- the next 5, port 42004 ( + the listener needs to be configured to listen on  42004 )
```